

EREDITARIETÀ E POLIMORFISMO

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



EREDITARIETÀ

- L'ereditarietà è un concetto centrale in OOP
- Una classe A è una **sottoclasse** di B (o B **superclasse** di A) quando
 - A eredita da B sia le sue variabili che i suoi metodi
 - Un'istanza di A è utilizzabile in ogni parte del codice in cui sia possibile utilizzare un'istanza di B
- La relazione di sottoclasse viene espressa con la keyword **extends**

```
1 public class A extends B {  
2     ...  
3 }
```

La classe A eredita tutti metodi e le variabili di B che possono essere acceduti o meno da A a seconda del loro modificatore di visibilità (*public*, *protected*, *private*)

- Ogni sottoclasse può avere a sua volta sottoclassi (gerarchia)
- Java non supporta ereditarietà multipla

EREDITARIETÀ E VARIABILI

- Ogni variabile della classe padre è anche una variabile della sottoclasse
- Variabili con lo stesso nome sono variabili **distinte**
- Possiamo accedere alle variabili della superclasse con la keyword **super**

```
1 public class B {  
2     protected String x = "B";  
3 }
```

```
1 public class A extends B {  
2     public String x = "A";  
3  
4     public void print() {  
5         System.out.println(x + " " + super.x);  
6     }  
7 }
```

La variabile *x* in *B* è accessibile con *super.x* in *A* poiché *protected*

EREDITARIETÀ E METODI

- Ogni metodo della classe padre è anche un metodo della sottoclasse
- Metodi con lo stesso nome sono **sovrascritti** (**overriding**)
- Possiamo comunque accedere al metodo della sottoclasse con **super**

```
1 public class B {  
2     protected String x = "B";  
3     public void print() {  
4         System.out.println(x);  
5     }  
6 }
```

```
1 public class A extends B {  
2     public String x = "A";  
3     public void print() { // Overriding  
4         System.out.println(x + " ");  
5         super.print(); // Metodo print() di B  
6     }  
7 }
```

EREDITARIETÀ E MODIFICATORI DI VISIBILITÀ

- Una variabile o metodo *private* non sono accessibili nella sottoclasse

```
1 public class B {  
2     private int x;           // Variabile privata  
3     private void print() { // Metodo privato  
4         System.out.println(x);  
5     }  
6 }
```

```
1 public class A extends B {  
2     public static void main(String args[]) {  
3         A a = new A():  
4         a.x = 100; // Errore  
5         a.print(); // Errore  
6     }  
7 }
```

EREDITARIETÀ E MODIFICATORI DI ACCESSO

- Una classe final non può essere estesa

```
1 public final class B {}
```

```
1 public class A extends B {} // Errore
```

- Un metodo final non può essere sovrascritto

```
1 public final class B {  
2     public final void print() {  
3         System.out.println("I am the B class");  
4     }  
5 }
```

```
1 public class A extends B {  
2     public void print() { // Errore  
3         System.out.println("I am the A class");  
4     }  
5 }
```

LA CLASSE OBJECT

- In Java la classe Object è di default superclasse di tutte le classi
- Utile per referenziare un oggetto di cui non conosciamo il tipo

```
1 Object x = new String("This is a string "); // Corretto
2 String y = new Object(); // Non corretto
```

- La classe Object espone alcuni metodi, ereditati da tutte le classi
 - *toString()*: ritorna una stringa descrittiva dell'oggetto
 - *equals(Object obj)*: verifica l'uguaglianza con un altro oggetto
 - *hashCode()*: ritorna un codice hash per l'oggetto
 - ...
- A volte è utile ridefinire (overriding) alcuni di questi metodi

IL METODO toString()

- Riassume in una stringa le informazioni di un oggetto
 - Nome della classe + codice hash dell'oggetto
- Viene invocato di default quando si passa un oggetto alla *println()*
- Potrebbe essere utile ridefinirlo per fare debugging o salvare log

```
1 public class A {  
2     public static void main(String args[]) {  
3         A a = new A();  
4         System.out.println(A); // toString di default  
5     }  
6 }
```

```
1 public class A {  
2     public String toString() { // Overriding  
3         return "I am the A class";  
4     }  
5     public static void main(String args[]) {  
6         A a = new A();  
7         System.out.println(a); // toString sovrascritto  
8     }  
9 }
```


IL METODO EQUALS()

- Metodo per confrontare l'uguaglianza di due oggetti

```
1 String A = new String("1");
2 String B = new String("1");
3 Integer C = Integer.valueOf(1);
4
5 System.out.println(A.equals(B)); // True
6 System.out.println(A.equals(C)); // False
```

- Attenzione: due oggetti *simili* non sono necessariamente *uguali*

```
1 public class Pair {
2     int x,y;
3     public Pair(int x, int y) {
4         this.x = x;
5         this.y = y;
6     }
7 }
```

```
1 Pair A = new Pair(1,2);
2 Pair B = new Pair(1,2);
3 System.out.println(A.equals(B)); // False
```

Necessario sovrascrivere *equals()* in *Pair* per confrontare il contenuto

COME SOVRASCRIVERE EQUALS()

- L'intestazione di *equals()*, deve essere la stessa che in [Object](#)

```
1 public class Pair {  
2     int x, y;  
3     public Pair(int x, int y) {  
4         this.x = x;  
5         this.y = y;  
6     }  
7  
8     public boolean equals(Object o) {  
9         if(o == null || !(o instanceof Pair))  
10            return false;  
11        else  
12            return this.x == ((Pair)o).x && this.y == ((Pair)o).y;  
13    }  
14 }
```

Typecasting a riga 12 necessario: il tipo *Object* non ha variabili *x*, *y*

- Adesso possiamo confrontare due oggetti di tipo *Pair* rispetto a *x*, *y*

```
1 Pair A = new Pair(1,2);  
2 Pair B = new Pair(1,2);  
3 System.out.println(A.equals(B)); // True: stesso contenuto  
4 System.out.println(A == B);      // False: oggetti differenti
```

- L'ereditarietà è strettamente legata al polimorfismo
- Letteralmente, polimorfismo vuol dire molte forme
- Il polimorfismo implica diverse *forme* al nome di un metodo
- La forma del metodo è determinata a runtime dalla JVM
- Conseguenza: un oggetto può comportarsi come se fosse di più tipi

ESEMPIO DI POLIMORFISMO

```
1 public class B {  
2     public String toString() {  
3         return "I am the B class";  
4     }  
5 }
```

```
1 public class A extends B {  
2     public String toString() {  
3         return "I am the A class";  
4     }  
5     public static void main(String args[]) {  
6         A a = new A();  
7         B b = new B();  
8         B c = new A(); // legale: A sottoclasse di B  
9         System.out.println(a); // Stampa "I am the A class"  
10        System.out.println(b); // Stampa "I am the B class"  
11        System.out.println(c); // Stampa "I am the A class"  
12    }  
13 }
```

A runtime la JVM determina per c il tipo A e quindi invoca toString() di A

ESERCIZIO

- Implementare in java una classe *Studente* che estenda la classe *Persona* (implementata precedentemente)
- La classe *Studente* deve includere le informazioni aggiuntive
 - Media (double, privato)
 - Esami sostenuti (intero, privato)
- Includere i nuovi metodi
 - *setNumExams()*: modifica il numero di esami sostenuti
 - *setAvgGrade()*: modifica la media degli esami sostenuti
 - *getNumExams()*: ritorna il numero di esami sostenuti
 - *getAvgGrade()*: ritorna la media degli esami sostenuti
- Sovrascrivere il metodo *getInfo()* con un nuovo metodo che stampi informazioni anche relativamente alla media e numero di esami sostenuti
 - Nota: potrebbe essere necessario modificare la classe *Persona*